

SWIGGY SALES DATA ANALYSIS

Complete Real-World SQL Case Study & Database Handbook

MySQL Implementation

Generated: June 28, 2026

Technology Stack: MySQL | Star Schema Architecture | Data Warehouse Design

This technical handbook compiles the end-to-end database infrastructure, rigorous data quality checks, relational schema design, and advanced business intelligence analytics queries executed during the Swiggy Sales Case Study project.

TABLE OF CONTENTS

1. Database Creation & Data Infrastructure
2. Data Quality Assurance, Validation & Cleaning
3. Dimensional Modeling (Star Schema Construction)
4. Key Performance Indicator (KPI) Computations
5. Deep-Dive Analytical Business Queries
6. Advanced Analytics & Insights

1. Database Creation & Data Infrastructure

Problem Statement: Establish a robust data foundation by creating a clean, well-structured database that can handle Swiggy's transactional order data with proper constraints and data types.

To start with a fully clean development environment, the database context is initialized and explicit verification checks read the loaded raw record counts from the baseline flat file import table.

Database Setup:

```
CREATE DATABASE IF NOT EXISTS swiggy_db;
USE swiggy_db;

CREATE TABLE IF NOT EXISTS swiggy_orders (
    State VARCHAR(100),
    City VARCHAR(100),
    Order_Date DATE,
    Restaurant_Name VARCHAR(255),
    Location VARCHAR(255),
    Category VARCHAR(100),
    Dish_Name VARCHAR(255),
    Price_INR DECIMAL(10,2),
    Rating FLOAT,
    Rating_Count INT
);
```

Data Loading:

```
SET GLOBAL local_infile = 1;

LOAD DATA LOCAL INFILE 'path/to/swiggy.csv'
INTO TABLE swiggy_orders
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES
(State, City, @raw_date, Restaurant_Name, Location,
 Category, Dish_Name, Price_INR, Rating, Rating_Count)
SET Order_Date = STR_TO_DATE(@raw_date, '%d-%m-%Y');
```

Key Implementation Details: The raw data is loaded into a staging table with proper character set encoding (UTF8MB4) to handle special characters. Date conversion is performed using STR_TO_DATE to transform the CSV date format into MySQL DATE type.

2. Data Quality Assurance, Validation & Cleaning

Problem Statement: Identify and eliminate data quality issues including NULL values, blank strings, and duplicate records to ensure the integrity of analytical insights.

2.1 Null Value Validation

A comprehensive matrix validation scans all attributes to guarantee that empty value representations do not cause skewing during statistical operations.

```
SELECT
    SUM(CASE WHEN state IS NULL THEN 1 ELSE 0 END) AS null_state,
    SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS null_city,
    SUM(CASE WHEN order_date IS NULL THEN 1 ELSE 0 END) AS null_order_date,
    SUM(CASE WHEN restaurant_name IS NULL THEN 1 ELSE 0 END) AS null_restaurant,
    SUM(CASE WHEN location IS NULL THEN 1 ELSE 0 END) AS null_location,
    SUM(CASE WHEN category IS NULL THEN 1 ELSE 0 END) AS null_category,
    SUM(CASE WHEN dish_name IS NULL THEN 1 ELSE 0 END) AS null_dish_name,
    SUM(CASE WHEN price_INR IS NULL THEN 1 ELSE 0 END) AS null_price,
    SUM(CASE WHEN rating IS NULL THEN 1 ELSE 0 END) AS null_rating,
    SUM(CASE WHEN rating_count IS NULL THEN 1 ELSE 0 END)
        AS null_rating_count
FROM swiggy_orders;
```

2.2 Blank/Empty String Validation

This verification detects categorical values initialized incorrectly with zero-length white-space characters during application entry logging.

```
SELECT * FROM swiggy_orders
WHERE state = ''
    OR city = ''
    OR restaurant_name = ''
    OR location = ''
    OR category = ''
    OR dish_name = '';
```

2.3 Comprehensive Row Duplicate Detection

Grouping constraints across every significant column identifies exact duplicate rows generated via tracking errors or pipeline ingestion anomalies.

```
SELECT
    state, city, order_date, restaurant_name, location,
    category, dish_name, price_INR, rating, rating_count,
    COUNT(*) AS duplicate_count
FROM swiggy_orders
GROUP BY
    state, city, order_date, restaurant_name, location,
    category, dish_name, price_INR, rating, rating_count
HAVING COUNT(*) > 1;
```

2.4 Duplicate Removal Execution

Using the analytical ROW_NUMBER() window function partitions rows across identical transaction properties and safely removes redundant duplicates while preserving unique baseline records.

```
ALTER TABLE swiggy_orders
ADD COLUMN id INT AUTO_INCREMENT PRIMARY KEY;

DELETE s
FROM swiggy_orders s
JOIN (
    SELECT id,
           ROW_NUMBER() OVER (
               PARTITION BY State, City, Order_Date, Restaurant_Name,
                           Location, Category, Dish_Name,
                           Price_INR, Rating, Rating_Count
               ORDER BY id
           ) AS rn
    FROM swiggy_orders
) d
ON s.id = d.id
WHERE d.rn > 1;
```

3. Dimensional Modeling (Star Schema Construction)

Problem Statement: Transform the normalized staging table into a star schema with dimension and fact tables to enable efficient analytical queries and maintain data warehouse best practices.

Transitioning from an unnormalized flat layout to a structured Star Schema reduces data repetition, enforces foreign key lookup constraints, and significantly speeds up query response times within presentation layers.

3.1 Dimension Table Creation

```
-- 1. Date Dimension Table
CREATE TABLE dim_date (
    date_id INT AUTO_INCREMENT PRIMARY KEY,
    full_date DATE,
    year INT,
    month INT,
    month_name VARCHAR(20),
    quarter INT,
    day INT,
    week_number INT
);

-- 2. Location Dimension Table
CREATE TABLE dim_location (
    location_id INT AUTO_INCREMENT PRIMARY KEY,
    state VARCHAR(100),
    city VARCHAR(100),
    location VARCHAR(200)
);

-- 3. Restaurant Dimension Table
CREATE TABLE dim_restaurant (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_name VARCHAR(200)
);

-- 4. Category Dimension Table
CREATE TABLE dim_category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category VARCHAR(100)
);

-- 5. Dish Dimension Table
CREATE TABLE dim_dish (
    dish_id INT AUTO_INCREMENT PRIMARY KEY,
    dish_name VARCHAR(200)
);
```

3.2 Fact Table Creation

```
CREATE TABLE fact_swiggy_orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
```

```

    date_id INT,
    location_id INT,
    restaurant_id INT,
    category_id INT,
    dish_id INT,
    price_INR DECIMAL(10,2),
    rating FLOAT,
    rating_count INT,

    CONSTRAINT FK_FactDate
        FOREIGN KEY (date_id)
        REFERENCES dim_date(date_id),

    CONSTRAINT FK_FactLocation
        FOREIGN KEY (location_id)
        REFERENCES dim_location(location_id),

    CONSTRAINT FK_FactRestaurant
        FOREIGN KEY (restaurant_id)
        REFERENCES dim_restaurant(restaurant_id),

    CONSTRAINT FK_FactCategory
        FOREIGN KEY (category_id)
        REFERENCES dim_category(category_id),

    CONSTRAINT FK_FactDish
        FOREIGN KEY (dish_id)
        REFERENCES dim_dish(dish_id)
);

```

3.3 Populating Dimension Tables

```

-- Populating dim_date
INSERT INTO dim_date (full_date, year, month, month_name, quarter, day, week_number)
SELECT DISTINCT
    order_date,
    YEAR(order_date),
    MONTH(order_date),
    MONTHNAME(order_date),
    QUARTER(order_date),
    DAY(order_date),
    WEEK(order_date)
FROM swiggy_orders
WHERE order_date IS NOT NULL;

-- Populating dim_location
INSERT INTO dim_location (state, city, location)
SELECT DISTINCT state, city, location
FROM swiggy_orders;

-- Populating dim_restaurant
INSERT INTO dim_restaurant (restaurant_name)
SELECT DISTINCT restaurant_name
FROM swiggy_orders;

-- Populating dim_category

```

```
INSERT INTO dim_category (category)
SELECT DISTINCT category
FROM swiggy_orders;
```

```
-- Populating dim_dish
INSERT INTO dim_dish (dish_name)
SELECT DISTINCT dish_name
FROM swiggy_orders;
```


3.4 Populating the Fact Table via Multi-Table Key Resolution

This integration join matches literal string records back into surrogate dimensional identification sequences to properly populate structural table keys and establish referential integrity.

```
INSERT INTO fact_swiggy_orders
  (date_id, location_id, restaurant_id, category_id, dish_id,
   price_INR, rating, rating_count)
SELECT
  dd.date_id,
  dl.location_id,
  dr.restaurant_id,
  dc.category_id,
  ds.dish_id,
  s.price_INR,
  s.rating,
  s.rating_count
FROM swiggy_orders s
INNER JOIN dim_date dd ON s.order_date = dd.full_date
INNER JOIN dim_location dl
  ON s.state = dl.state AND s.city = dl.city
  AND s.location = dl.location
INNER JOIN dim_restaurant dr ON s.restaurant_name = dr.restaurant_name
INNER JOIN dim_category dc ON s.category = dc.category
INNER JOIN dim_dish ds ON s.dish_name = ds.dish_name;
```

3.5 Schema Validation Join Blueprint

A structural integrity drill-down test verifies that dimensions and relational joins function accurately, matching the full historical log row volume counts.

```
SELECT * FROM fact_swiggy_orders f
JOIN dim_date d ON f.date_id = d.date_id
JOIN dim_location l ON f.location_id = l.location_id
JOIN dim_restaurant r ON f.restaurant_id = r.restaurant_id
JOIN dim_category c ON f.category_id = c.category_id
JOIN dim_dish ds ON f.dish_id = ds.dish_id;
```

4. Key Performance Indicator (KPI) Computations

Problem Statement: Calculate critical business metrics that display overall system operational health and provide immediate visibility into key business drivers.

These primary financial metrics display general system operational health directly inside operational status dashboards, enabling rapid decision-making and performance monitoring.

KPI 1: Total Orders

```
SELECT COUNT(*) AS total_orders
FROM fact_swiggy_orders;
```

Purpose: Measures the total volume of orders processed in the system. This metric indicates market penetration, platform usage intensity, and operational scale.

KPI 2: Total Revenue (INR Millions)

```
SELECT CONCAT(
    FORMAT(SUM(price_INR) / 1000000, 2),
    ' INR Million'
) AS total_revenue
FROM fact_swiggy_orders;
```

Purpose: Calculates aggregate monetary value across all transactions. Normalized to millions for executive reporting. Essential for understanding platform profitability and financial performance.

KPI 3: Average Dish Price

```
SELECT CONCAT(
    FORMAT(AVG(price_INR), 2),
    ' INR'
) AS average_dish_price
FROM fact_swiggy_orders;
```

Purpose: Determines the typical transaction value across the platform. Insights into customer spending patterns, pricing strategy effectiveness, and market positioning.

KPI 4: Average Order Rating

```
SELECT AVG(rating) AS average_rating
FROM fact_swiggy_orders;
```

Purpose: Aggregates customer satisfaction metrics across all orders. Directly correlates with retention rates, brand perception, and operational quality standards.

5. Deep-Dive Analytical Business Queries

Problem Statement: Extract granular business insights across temporal, geographic, and product dimensions to identify high-performing regions, optimize inventory, and enhance customer acquisition strategies.

5.1 Temporal Analysis Series

Monthly Volume & Revenue Order Patterns

```
SELECT
    d.year,
    d.month,
    d.month_name,
    COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_date d ON f.date_id = d.date_id
GROUP BY d.year, d.month, d.month_name
ORDER BY total_orders DESC;
```

Insight: Identifies seasonal trends and monthly performance variations. Critical for demand forecasting, promotional calendar planning, and resource allocation.

Day-of-Week Operational Performance

```
SELECT
    DAYNAME(d.full_date) AS day_of_week,
    COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_date d ON f.date_id = d.date_id
GROUP BY
    DAYNAME(d.full_date),
    DAYOFWEEK(d.full_date)
ORDER BY
    DAYOFWEEK(d.full_date);
```

Insight: Reveals weekly consumption patterns. Weekends typically show higher volumes. Enables targeted promotions and operational planning.

5.2 Location-Based Analysis Series

Top 10 High-Volume Consumer Cities

```
SELECT
    l.city,
    COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_location l ON f.location_id = l.location_id
GROUP BY l.city
ORDER BY total_orders DESC
LIMIT 10;
```

Insight: Identifies metropolitan markets with highest demand. Strategic importance for expansion planning, supply chain optimization, and marketing spend allocation.

Regional State Financial Performance Metrics

```
SELECT
    l.state,
    SUM(f.price_INR) AS total_revenue
FROM fact_swiggy_orders f
JOIN dim_location l ON f.location_id = l.location_id
GROUP BY l.state
ORDER BY total_revenue DESC
LIMIT 10;
```

Insight: Revenue contribution by state-level regions. Guides investment decisions, regulatory compliance focus, and state-specific business strategies.

5.3 Food Performance Analysis Series

Top 10 Restaurant Brands by Order Volume

```
SELECT
    r.restaurant_name,
    COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_restaurant r ON f.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_name
ORDER BY total_orders DESC
LIMIT 10;
```

Insight: Identifies market leaders by transaction volume. Performance indicators for restaurant partner strategy and exclusive collaboration opportunities.

Top Categories by Order Volume

```
SELECT
    c.category,
    COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_category c ON f.category_id = c.category_id
GROUP BY c.category
ORDER BY total_orders DESC;
```

Insight: Cuisine category popularity rankings. Critical for vendor onboarding, menu curation, and category-specific marketing campaigns.

Top 10 Most Frequently Purchased Items

```
SELECT
    ds.dish_name,
    COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_dish ds ON f.dish_id = ds.dish_id
GROUP BY ds.dish_name
ORDER BY total_orders DESC
LIMIT 10;
```

Insight: Best-selling menu items across the platform. Information for restaurant recommendations, inventory planning, and viral marketing opportunities.

5.4 Consumer Spending & Quality Analysis

Specialized Consumer Spending Profiling (Price Buckets)

```
SELECT
    CASE
        WHEN price_INR < 100 THEN 'Under 100'
        WHEN price_INR BETWEEN 100 AND 199 THEN '100-199'
        WHEN price_INR BETWEEN 200 AND 299 THEN '200-299'
        WHEN price_INR BETWEEN 300 AND 499 THEN '300-499'
        ELSE '500+'
    END
```

```

        END AS customer_spend_bucket,
        COUNT(*) AS total_orders
    FROM fact_swiggy_orders
    GROUP BY
        CASE
            WHEN price_INR < 100 THEN 'Under 100'
            WHEN price_INR BETWEEN 100 AND 199 THEN '100-199'
            WHEN price_INR BETWEEN 200 AND 299 THEN '200-299'
            WHEN price_INR BETWEEN 300 AND 499 THEN '300-499'
            ELSE '500+'
        END
    ORDER BY total_orders DESC;

```

Insight: Segmentation of customer spending behavior. Reveals price sensitivity, market composition, and opportunities for premium product lines.

Distribution Analysis of Ratings

```

SELECT
    ROUND(rating) AS rating_bucket,
    COUNT(*) AS order_count
FROM fact_swiggy_orders
GROUP BY ROUND(rating)
ORDER BY rating_bucket DESC;

```

Insight: Quality perception distribution. Helps identify systematic quality issues and measure overall satisfaction trends.

6. Advanced Analytics & Insights

Problem Statement: Conduct deep correlation analyses between revenue, ratings, and pricing to uncover strategic opportunities for business optimization and competitive advantage.

Revenue vs Rating Analysis

```
SELECT
  CASE
    WHEN rating < 4.0 THEN 'Below 4.0'
    WHEN rating BETWEEN 4.0 AND 4.49 THEN '4.0 - 4.49'
    WHEN rating BETWEEN 4.5 AND 4.79 THEN '4.5 - 4.79'
    ELSE '4.8+'
  END AS rating_group,
  COUNT(*) AS total_orders,
  ROUND(SUM(price_INR),2) AS revenue,
  ROUND(AVG(price_INR),2) AS avg_price
FROM fact_swiggy_orders
GROUP BY rating_group;
```

Key Question: Do highly-rated dishes drive more revenue? This analysis reveals correlation between customer satisfaction and willingness to spend.

Top Restaurants by Average Rating

```
SELECT
  r.restaurant_name,
  ROUND(AVG(f.rating),2) AS avg_rating,
  COUNT(*) AS total_orders
FROM fact_swiggy_orders f
JOIN dim_restaurant r ON f.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_name
HAVING COUNT(*) >= 50
ORDER BY avg_rating DESC
LIMIT 10;
```

Key Question: Which restaurants consistently delight customers? Identifies quality leaders for exclusive partnerships and customer loyalty programs.

Price vs Rating Relationship

```
SELECT
  CASE
    WHEN price_INR < 100 THEN 'Under 100'
    WHEN price_INR BETWEEN 100 AND 199 THEN '100-199'
    WHEN price_INR BETWEEN 200 AND 299 THEN '200-299'
    WHEN price_INR BETWEEN 300 AND 499 THEN '300-499'
    ELSE '500+'
  END AS price_bucket,
  ROUND(AVG(rating),2) AS avg_rating
FROM fact_swiggy_orders
GROUP BY price_bucket;
```

Key Question: Do expensive dishes get better ratings? This reveals whether premium pricing is justified by quality perception.

Category Revenue Share

```
SELECT
    c.category,
    ROUND(
        SUM(f.price_INR) * 100 /
        (SELECT SUM(price_INR) FROM fact_swiggy_orders),
        2
    ) AS revenue_percentage
FROM fact_swiggy_orders f
JOIN dim_category c ON f.category_id = c.category_id
GROUP BY c.category
ORDER BY revenue_percentage DESC;
```

Insight: Portfolio contribution by food category. Enables prioritized resource allocation and category-specific growth strategies.

Restaurant Efficiency Score Analysis

```
SELECT
    r.restaurant_name,
    COUNT(*) AS orders,
    ROUND(AVG(f.rating),2) AS avg_rating,
    ROUND(SUM(f.price_INR),2) AS revenue
FROM fact_swiggy_orders f
JOIN dim_restaurant r ON f.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_name
HAVING COUNT(*) >= 50
ORDER BY revenue DESC;
```

Metric Definition: High orders + High ratings + High revenue = Top-performing restaurants. Composite measure for identifying partner success stories and replicable operational models.

State-wise Average Rating Performance

```
SELECT
    l.state,
    ROUND(AVG(f.rating),2) AS avg_rating
FROM fact_swiggy_orders f
JOIN dim_location l ON f.location_id = l.location_id
GROUP BY l.state
ORDER BY avg_rating DESC;
```

Insight: Geographic quality perception variations. Identifies regions requiring operational improvements and opportunities for quality-driven competitive advantages.

Top Revenue per Order Categories

```
SELECT
    c.category,
    ROUND(SUM(f.price_INR)/COUNT(*),2) AS revenue_per_order
FROM fact_swiggy_orders f
JOIN dim_category c ON f.category_id = c.category_id
```



```
GROUP BY c.category  
ORDER BY revenue_per_order DESC;
```

Insight: Average transaction value by cuisine type. Identifies high-margin categories and pricing optimization opportunities.

Conclusion & Key Takeaways

This comprehensive SQL analysis framework provides actionable business intelligence across temporal, geographic, operational, and qualitative dimensions. The star schema architecture enables:

- Rapid query execution for real-time dashboard updates
- Scalable dimensional model supporting future expansion
- Referential integrity through foreign key constraints
- Efficient aggregation for hierarchical roll-ups
- Separation of fact (measurements) from dimensions (context)

Implementation Notes: The MySQL syntax used throughout this handbook is fully compatible with modern relational database management systems. For production deployments, consider implementing indexes on foreign key columns, creating materialized views for frequently accessed aggregations, and establishing partitioning strategies for the fact table to optimize performance as data volumes scale.

Recommended Next Steps: Integrate this data warehouse with business intelligence tools (Tableau, Power BI, Looker) for interactive visualizations, establish automated ETL pipelines for continuous data refresh, implement row-level security for confidential restaurant metrics, and conduct regular query optimization reviews to maintain performance standards.